

```
In [1]: import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [2]: df=pd.read_csv ('/kaggle/input/diamonds/diamonds.csv')
```

```
In [3]: df
```

```
Out[3]:
```

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
0	1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...	...
53935	53936	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	53937	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	53938	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	53939	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	53940	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 11 columns

```
In [4]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   53940 non-null  int64
1   carat        53940 non-null  float64
2   cut          53940 non-null  object
3   color        53940 non-null  object
4   clarity      53940 non-null  object
5   depth        53940 non-null  float64
6   table        53940 non-null  float64
7   price        53940 non-null  int64
8   x            53940 non-null  float64
9   y            53940 non-null  float64
10  z            53940 non-null  float64
dtypes: float64(6), int64(2), object(3)
memory usage: 4.5+ MB

```

In [5]: `df.shape`

Out[5]: (53940, 11)

In [6]: `df.describe()`

Out[6]:

	Unnamed: 0	carat	depth	table	price	x	
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.
mean	26970.500000	0.797940	61.749405	57.457184	3932.799722	5.731157	5.
std	15571.281097	0.474011	1.432621	2.234491	3989.439738	1.121761	1.
min	1.000000	0.200000	43.000000	43.000000	326.000000	0.000000	0.
25%	13485.750000	0.400000	61.000000	56.000000	950.000000	4.710000	4.
50%	26970.500000	0.700000	61.800000	57.000000	2401.000000	5.700000	5.
75%	40455.250000	1.040000	62.500000	59.000000	5324.250000	6.540000	6.
max	53940.000000	5.010000	79.000000	95.000000	18823.000000	10.740000	58.

In [8]: `df.head(10)`

```
Out[8]:
```

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
0	1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	6	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	7	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	8	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	9	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	10	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39

```
In [9]: df.tail()
```

```
Out[9]:
```

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
53935	53936	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	53937	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	53938	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	53939	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	53940	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

```
In [10]: df.tail(9)
```

```
Out[10]:
```

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
53931	53932	0.71	Premium	F	SI1	59.8	62.0	2756	5.74	5.73	3.43
53932	53933	0.70	Very Good	E	VS2	60.5	59.0	2757	5.71	5.76	3.47
53933	53934	0.70	Very Good	E	VS2	61.2	59.0	2757	5.69	5.72	3.49
53934	53935	0.72	Premium	D	SI1	62.7	59.0	2757	5.69	5.73	3.58
53935	53936	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	53937	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	53938	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	53939	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	53940	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

```
In [11]: df['cut']
```

```
Out[11]: 0          Ideal
         1          Premium
         2           Good
         3          Premium
         4           Good
         ...
        53935         Ideal
        53936          Good
        53937    Very Good
        53938          Premium
        53939         Ideal
        Name: cut, Length: 53940, dtype: object
```

```
In [12]: df['cut'].head(20)
```

```
Out[12]: 0          Ideal
         1          Premium
         2           Good
         3          Premium
         4           Good
         5    Very Good
         6    Very Good
         7    Very Good
         8           Fair
         9    Very Good
        10          Good
        11          Ideal
        12          Premium
        13          Ideal
        14          Premium
        15          Premium
        16          Ideal
        17           Good
        18           Good
        19    Very Good
        Name: cut, dtype: object
```

```
In [14]: df[['cut', 'color']]
```

Out[14]:

	cut	color
0	Ideal	E
1	Premium	E
2	Good	E
3	Premium	I
4	Good	J
...	...	...
53935	Ideal	D
53936	Good	D
53937	Very Good	D
53938	Premium	H
53939	Ideal	D

53940 rows × 2 columns

In [15]: `df[['cut', 'color']].head(20)`

Out[15]:

	cut	color
0	Ideal	E
1	Premium	E
2	Good	E
3	Premium	I
4	Good	J
5	Very Good	J
6	Very Good	I
7	Very Good	H
8	Fair	E
9	Very Good	H
10	Good	J
11	Ideal	J
12	Premium	F
13	Ideal	J
14	Premium	E
15	Premium	E
16	Ideal	I
17	Good	J
18	Good	J
19	Very Good	J

In [16]: `df[['cut','color','price']].head(20)`

Out[16]:

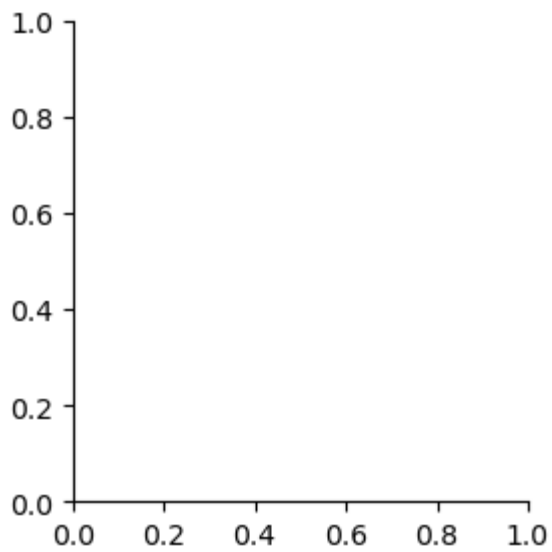
	cut	color	price
0	Ideal	E	326
1	Premium	E	326
2	Good	E	327
3	Premium	I	334
4	Good	J	335
5	Very Good	J	336
6	Very Good	I	336
7	Very Good	H	337
8	Fair	E	337
9	Very Good	H	338
10	Good	J	339
11	Ideal	J	340
12	Premium	F	342
13	Ideal	J	344
14	Premium	E	345
15	Premium	E	345
16	Ideal	I	348
17	Good	J	351
18	Good	J	351
19	Very Good	J	351

In [17]: `df[['cut','color','x','y','z']].head(20)`

Out[17]:

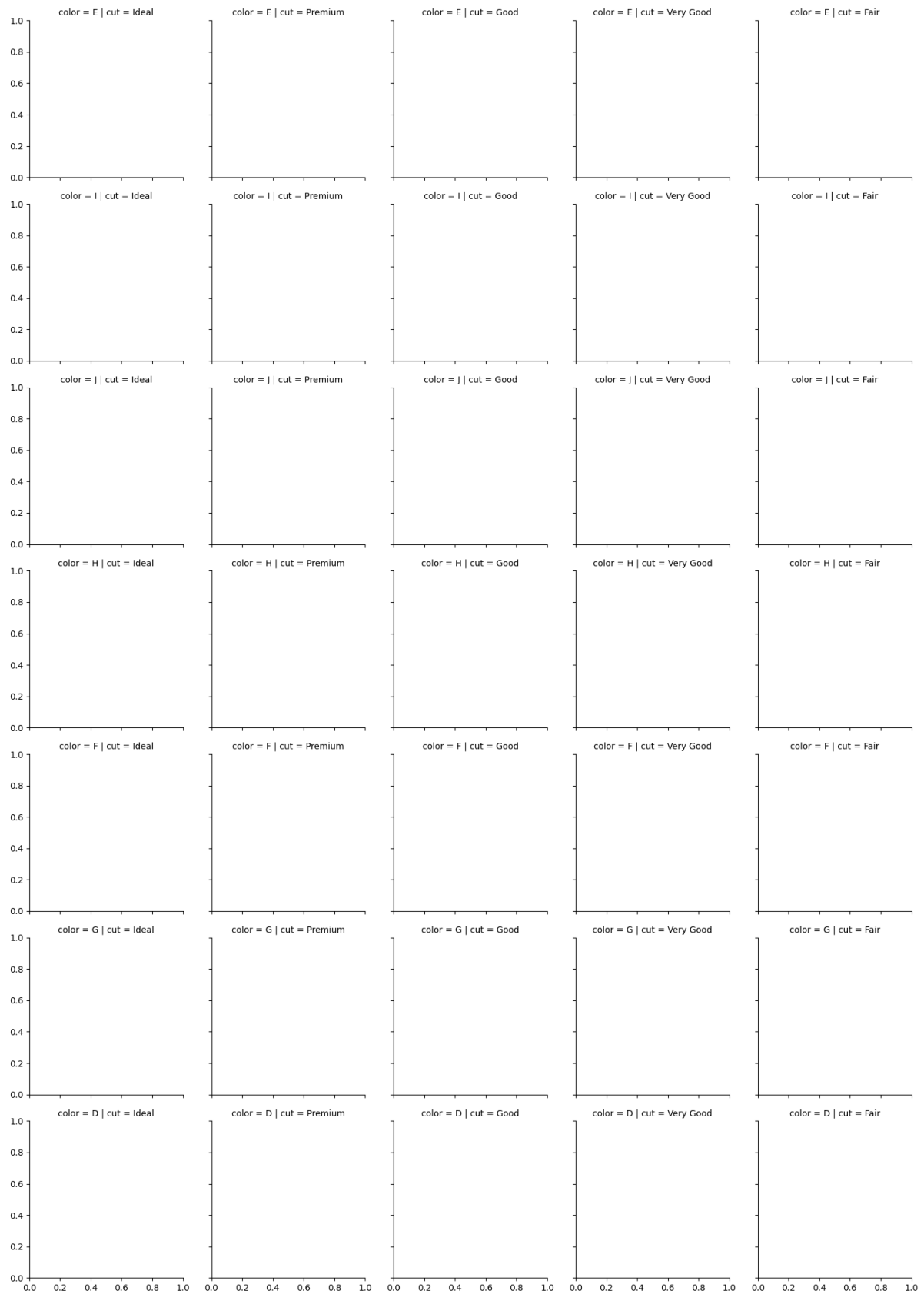
	cut	color	x	y	z
0	Ideal	E	3.95	3.98	2.43
1	Premium	E	3.89	3.84	2.31
2	Good	E	4.05	4.07	2.31
3	Premium	I	4.20	4.23	2.63
4	Good	J	4.34	4.35	2.75
5	Very Good	J	3.94	3.96	2.48
6	Very Good	I	3.95	3.98	2.47
7	Very Good	H	4.07	4.11	2.53
8	Fair	E	3.87	3.78	2.49
9	Very Good	H	4.00	4.05	2.39
10	Good	J	4.25	4.28	2.73
11	Ideal	J	3.93	3.90	2.46
12	Premium	F	3.88	3.84	2.33
13	Ideal	J	4.35	4.37	2.71
14	Premium	E	3.79	3.75	2.27
15	Premium	E	4.38	4.42	2.68
16	Ideal	I	4.31	4.34	2.68
17	Good	J	4.23	4.29	2.70
18	Good	J	4.23	4.26	2.71
19	Very Good	J	4.21	4.27	2.66

In [18]: `sns.FacetGrid(df)`Out[18]: `<seaborn.axisgrid.FacetGrid at 0x7fbe4dacba90>`



```
In [19]: sns.FacetGrid(df, col='cut', row='color')
```

```
Out[19]: <seaborn.axisgrid.FacetGrid at 0x7fbe4a96a4d0>
```



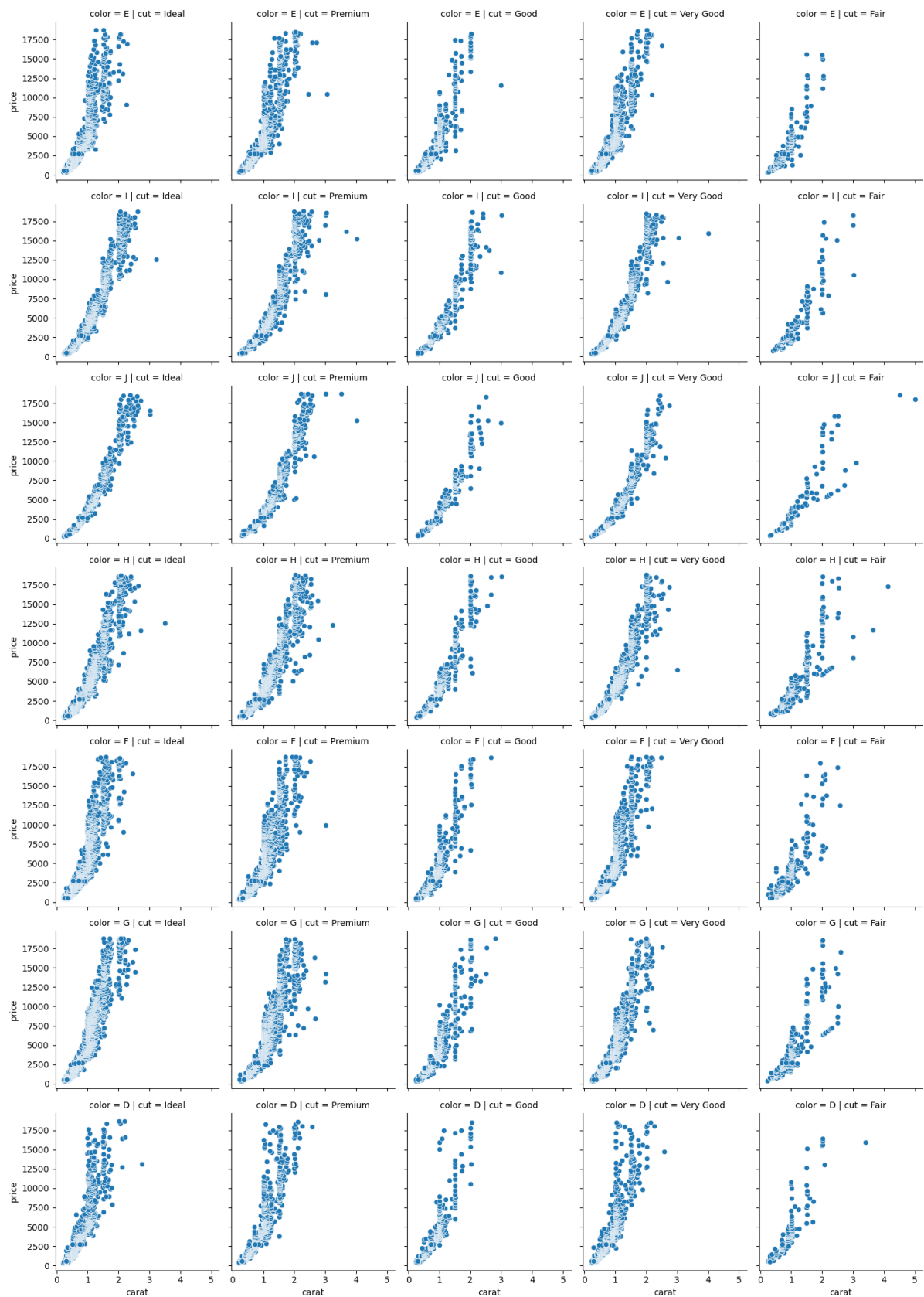
```
In [20]: g = sns.FacetGrid(df, col='cut', row='color')
```

```
Out[20]: <seaborn.axisgrid.FacetGrid at 0x7fbe4e0e4f90>
```



```
In [22]: g = sns.FacetGrid(df, col='cut', row='color')
g.map(sns.scatterplot, "carat", "price")
```

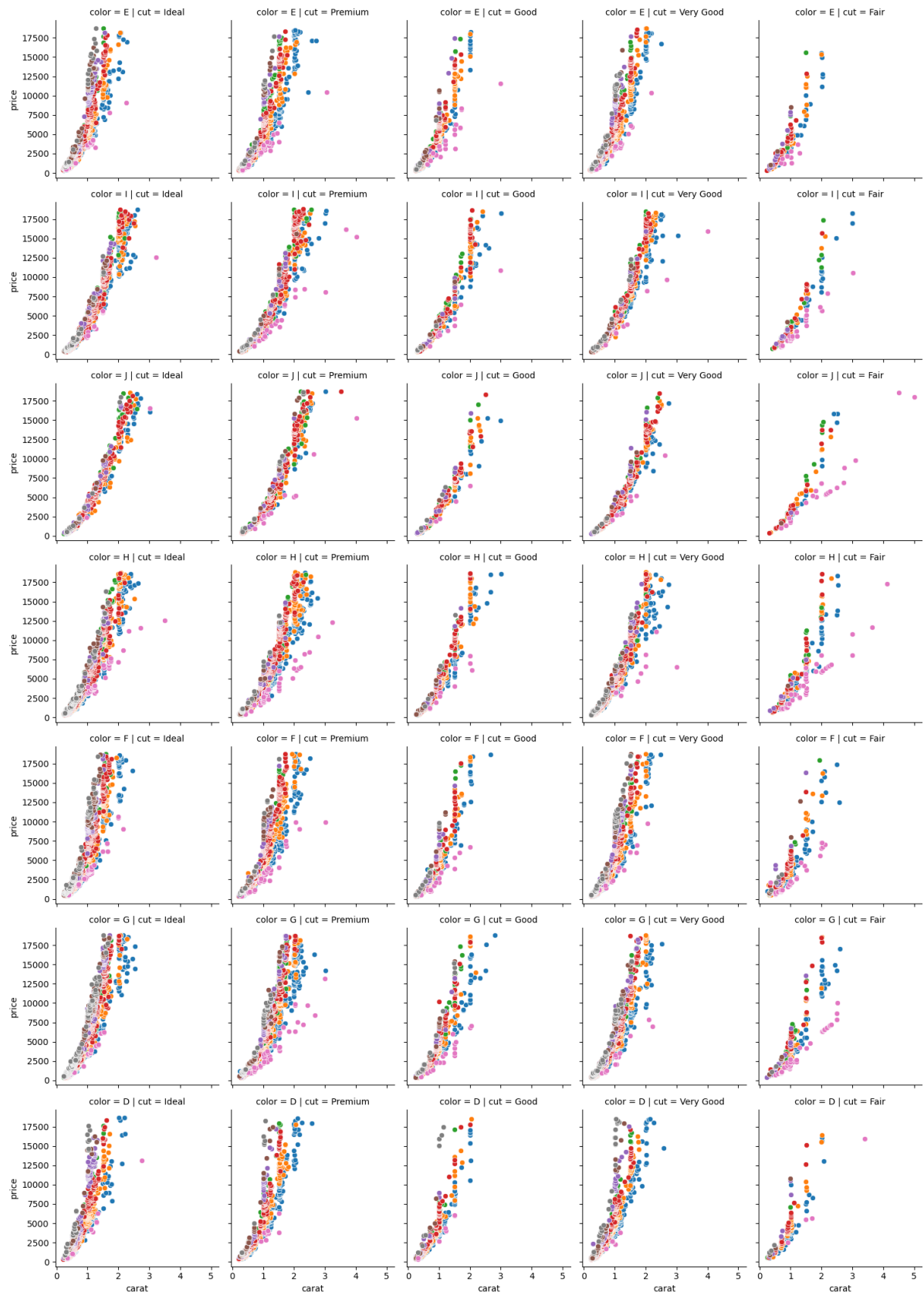
```
Out[22]: <seaborn.axisgrid.FacetGrid at 0x7fbe3bad9390>
```



```
In [24]: g = sns.FacetGrid(df, col='cut', row='color', hue='clarity')
g.map(sns.scatterplot, "carat", "price")

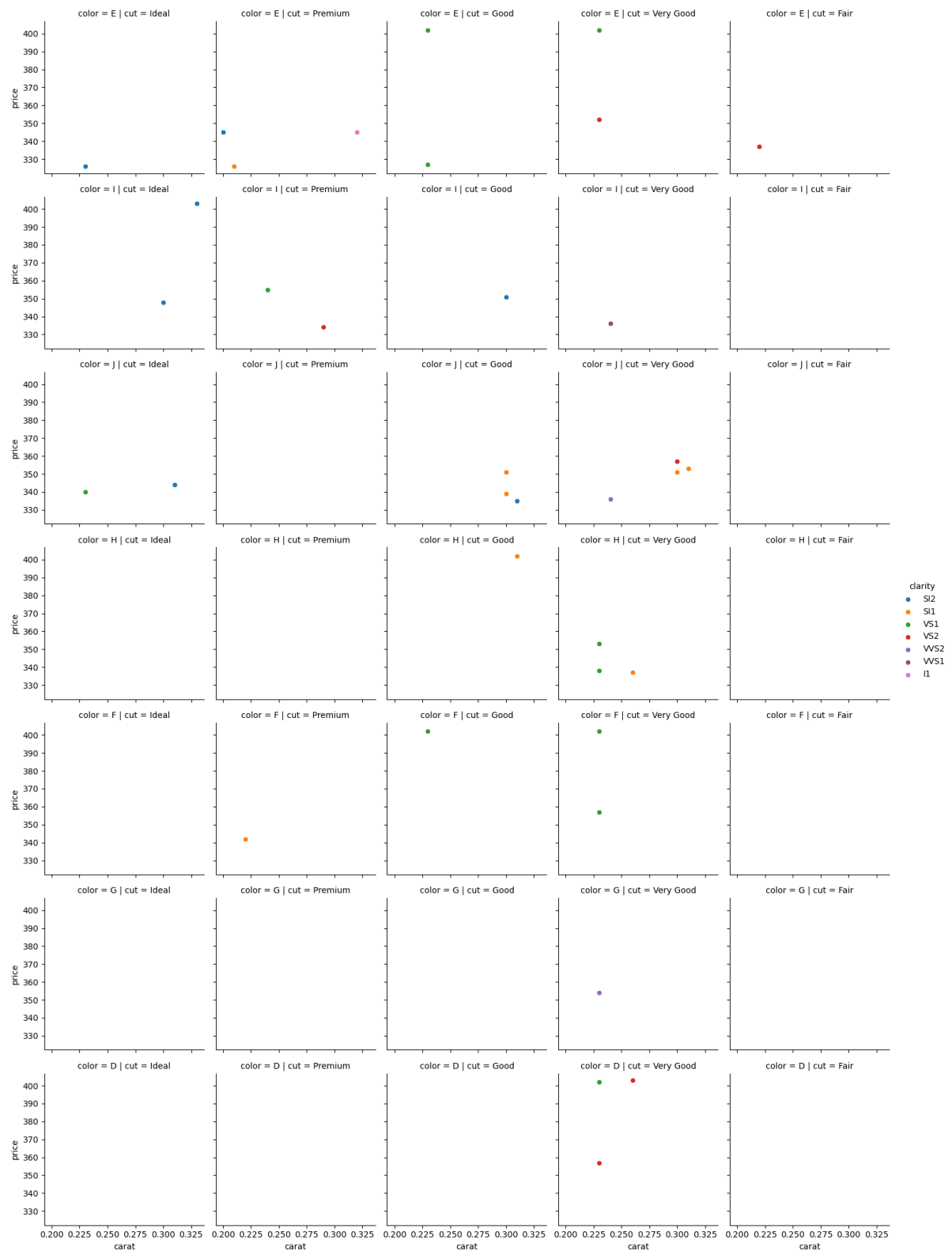
#g = sns.FacetGrid(df, col='cut', row='color')
#g.map(sns.scatterplot, "carat", "price")
```

Out[24]: <seaborn.axisgrid.FacetGrid at 0x7fbe3abf42d0>



```
In [28]: newdf=df.head(40)
g = sns.FacetGrid(newdf, col='cut', row='color', hue='clarity')
g.map(sns.scatterplot, "carat", "price")
g.add_legend()
```

Out[28]: <seaborn.axisgrid.FacetGrid at 0x7fbe36b648d0>

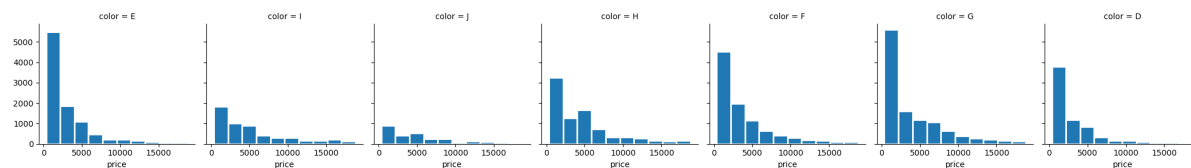


```
In [29]: newdf=df.head(40)
g = sns.FacetGrid(newdf, col='cut', row='color', hue='clarity')
g.map(sns.histplot, "carat")
g.add_legend()
```

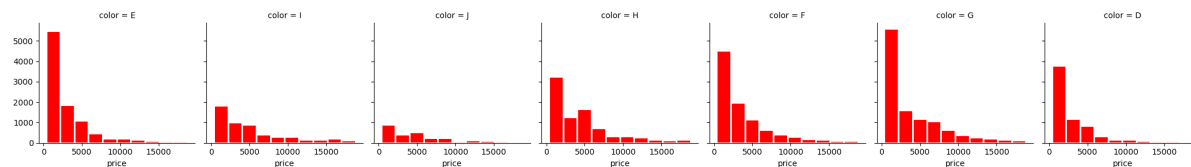
Out[29]: <seaborn.axisgrid.FacetGrid at 0x7fbe387c2610>



```
In [30]: g = sns.FacetGrid(df, col="color")
g = g.map(plt.hist, "price", edgecolor='w', linewidth=2)
```

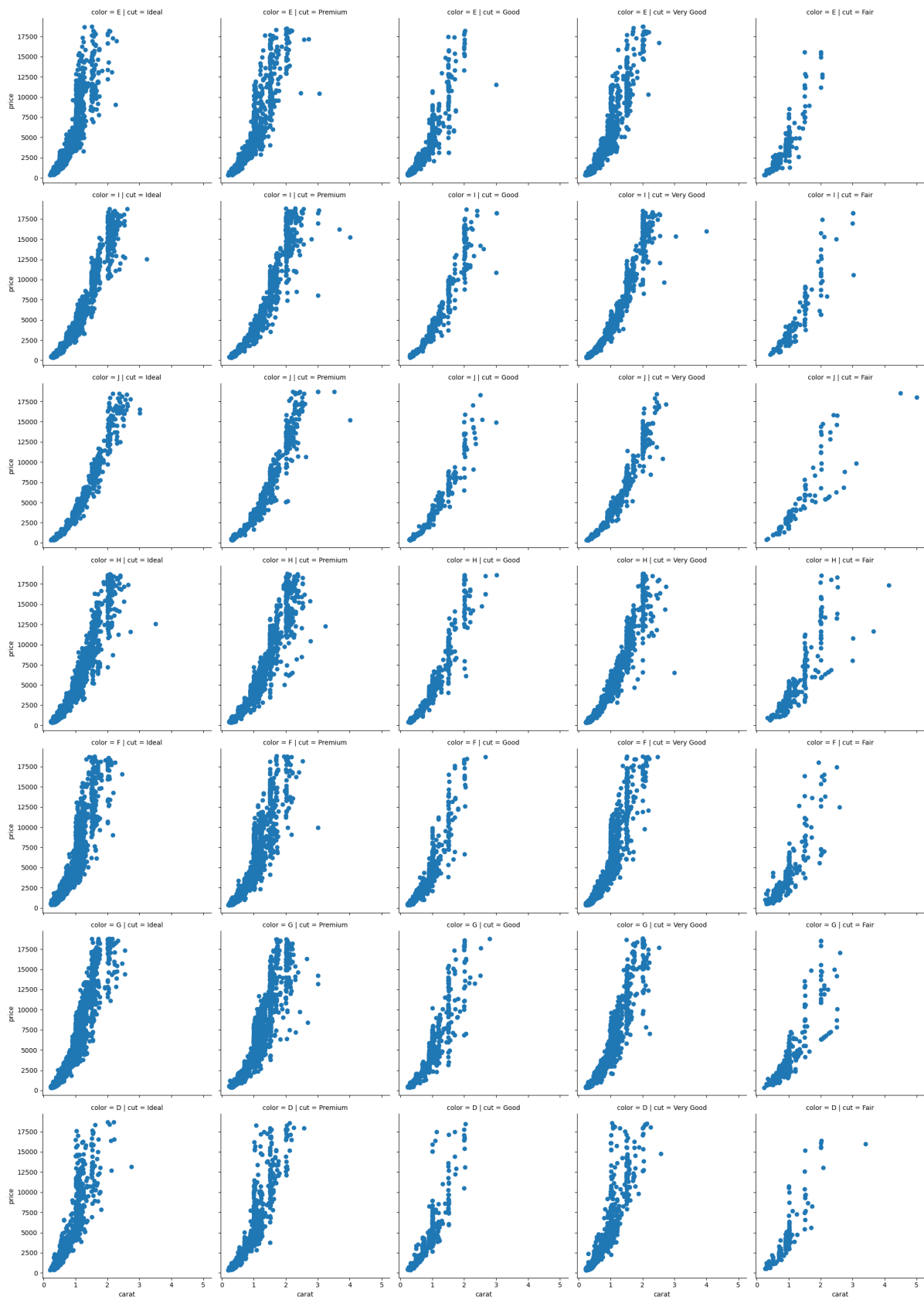


```
In [18]: g = sns.FacetGrid(df, col="color")
g = g.map(plt.hist, "price", edgecolor='w', linewidth=2, color='#ff0000')
```



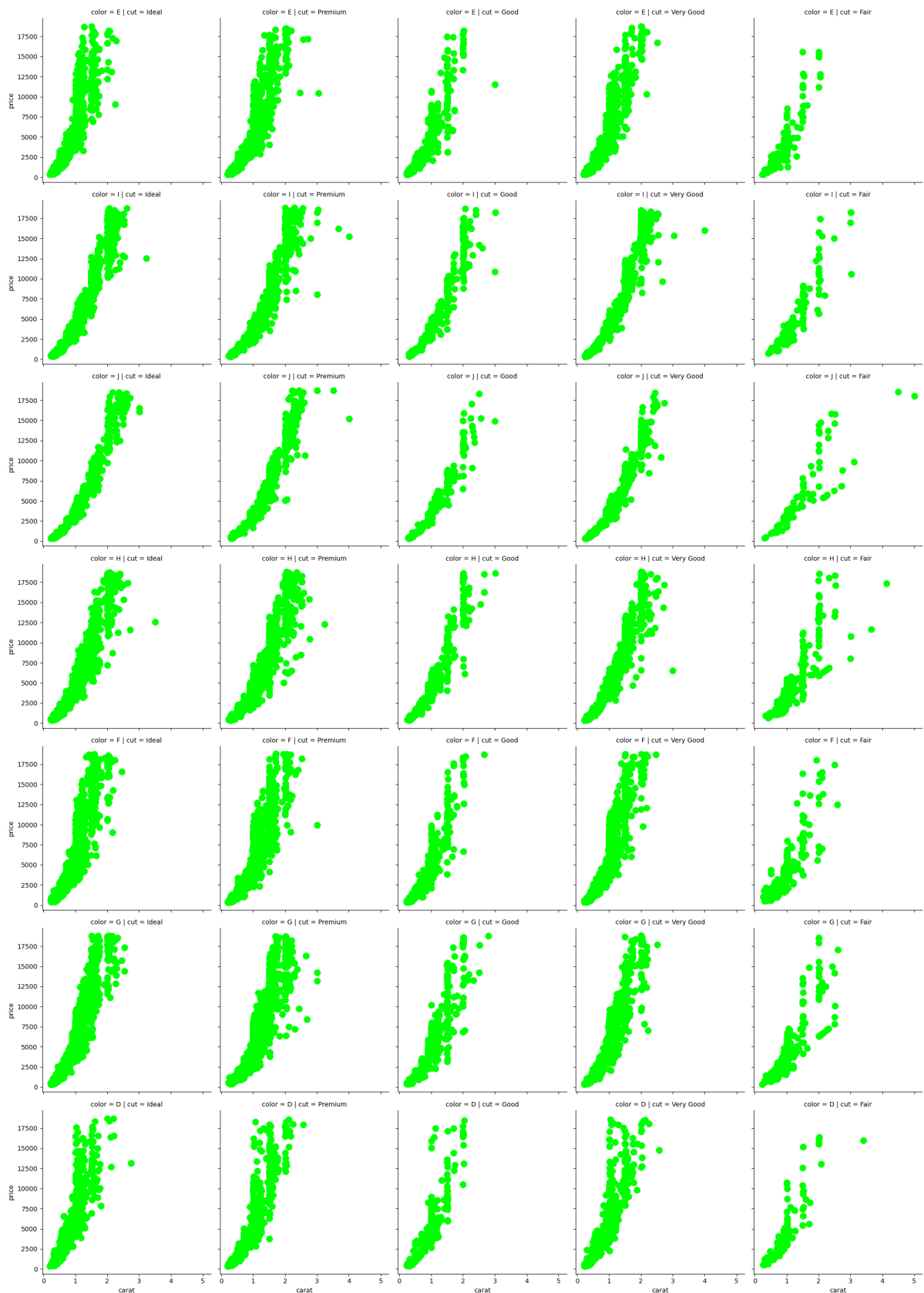
```
In [19]: g = sns.FacetGrid(df, col='cut', row='color', height=4, aspect=1)
g.map(plt.scatter, "carat", "price")
```

```
Out[19]: <seaborn.axisgrid.FacetGrid at 0x7f39272a0810>
```

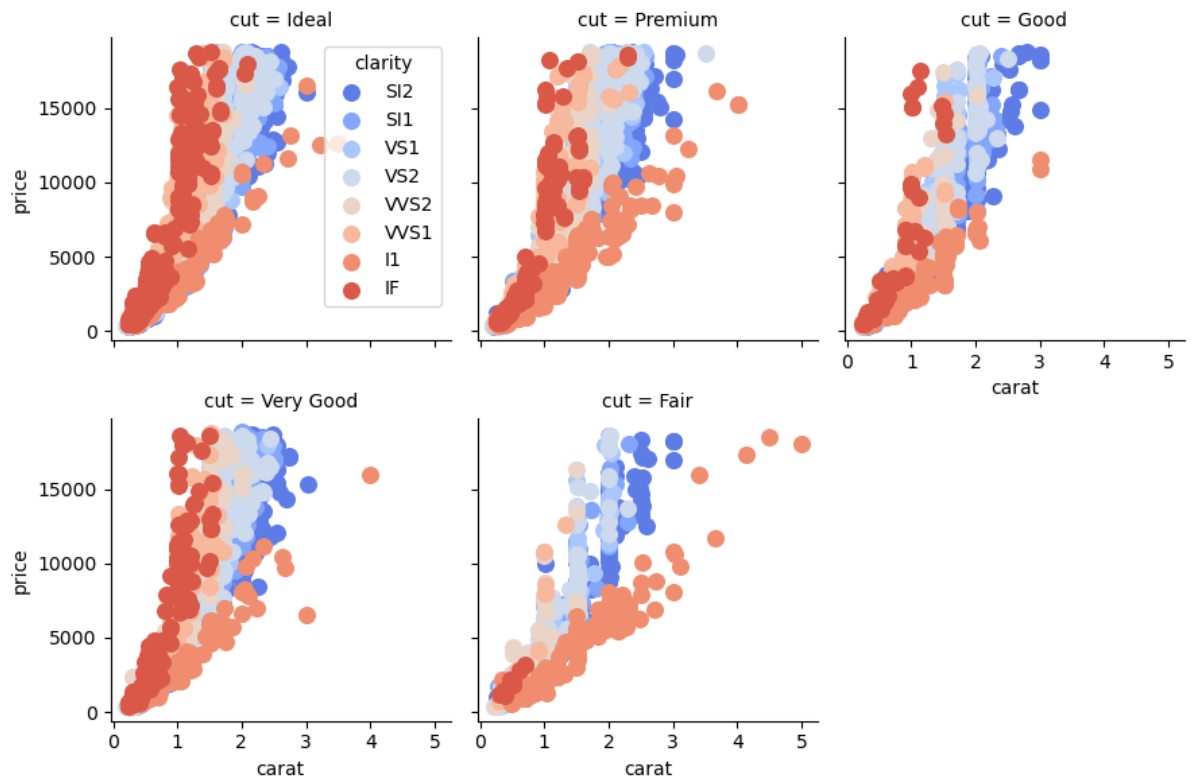
```
In [20]: g = sns.FacetGrid(df, col='cut', row='color', height=4, aspect=1)
g.map(plt.scatter, "carat", "price", s=90, color = '#00ff00')
```

```
Out[20]: <seaborn.axisgrid.FacetGrid at 0x7f3924e6bf90>
```



```
In [21]: g = sns.FacetGrid(df, col='cut', hue='clarity', palette='coolwarm', col_wrap=3, leg
g.map(plt.scatter, "carat", "price", s=70)
g.add_legend()
```

```
Out[21]: <seaborn.axisgrid.FacetGrid at 0x7f3927c9bad0>
```



```
In [23]: res = sns.scatterplot(x="carat", y="price", data=df)
plt.show()
```

